

```
In [ ]: #1. Two Sum Problem (Using a Dictionary)
class solution:
    def TwoSum(self,arr:list[int],target:int)->list[int]:
        dict1={}

        for i in range(len(arr)):
            rem=target-arr[i]

            if rem in dict1:
                return [dict1[rem],i]
            dict1[arr[i]]=i
obj=solution()
result=obj.TwoSum([5,1,8,2,6,4,9],12)
print(f"REsult is:{result}")
```

REsult is:[2, 5]

```
In [ ]: #Intersection of 2 arrays
# 2. Intersection of Two Arrays (Using Sets)
# The goal is to find common unique elements between two lists.
# Step 1: Convert both input lists into Python Sets. This automatically removes
# .
# Step 2: Use the intersection operator (&) or the .intersection() method to find
# .
# Step 3: Convert the resulting set back into a list to match the required output

class solution:
    def intersection(self,l1:list[int],l2:list[int])->list[int]:
        s1=set(l1)
        s2=set(l2)

        return list(s1.intersection(s2))
obj=solution()
r=obj.intersection([1,2,2,1],[2,2])
print(f'Result is:{r}')
```

Result is:[2]

```
In [3]: #Fisr Unique Character
class solution:
    def FirstUnique(self,s:str)->int:
        freq={}

        for i in s:
            if i not in freq:
                freq[i]=1
            else:
                freq[i]+=1

        for i in range(len(s)):
            if freq[s[i]]==1:
                return i
        return -1
obj=solution()
r=obj.FirstUnique("MALAYALAM")
print(f'Result is:{r}')
```

Result is:4

```
In [13]: #ANAGRAM
class solution:
    def ValidAnagram(self,s1:str,s2:str)->bool:

        freq={}
        for i in s1:
            if i not in freq:
                freq[i]=1
            else:
                freq[i]+=1
        for i in s2:
            if i not in freq:
                return False
            else:
                freq[i]-=1
        for i in freq.values():
            if i==0:
                return True

            return False
obj=solution()
r=obj.ValidAnagram("anagram","nagaram")
print(f'Result is:{r}')
```

Result is:True

```
In [ ]: #Substring of size 3 with distiinct character
class solution:
    def GoodSubString(self,s:str)->int:
        ans=0
        n=len(s)
        for i in range(n-2):
            if s[i]!=s[i+1] and s[i+1]!=s[i+2] and s[i+2]!=s[i]:
                ans+=1
        return ans
obj=solution()
r=obj.GoodSubString("aababcabc")
print(f'Result is:{r}')
```

Result is:4

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](https://aka.ms/vscodeJupyterKernelCrash) for more info.

View Jupyter [log](command:jupyter.viewOutput) for further details.

In []: